BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO ASSIGNMENT 01

PHÁT TRIỂN HỆ THỐNG THÔNG MINH

Giảng viên: **Trần Đình Quế**

Sinh viên: **Nguyễn Tuấn Tài**

Mã sinh viên: **B23DCCN731**

Nhóm lớp: **06**

Hà Nội – 2026

Mục lục

I: TỔNG QUAN LÝ THUYẾT 4
1. Khái niệm 4
2. Phân biệt Bản chất 4
3. Chu trình 5 Bước Phát triển Hệ thống Thông minh 5
4. Vai trò Cốt lõi của Biểu diễn Tri thức 5
5. Vị trí của Học máy Truyền thống trong Lịch sử AI 6
6. Cơ sở Lý thuyết về Đánh giá và Đo lường Hiệu năng 6
6.1. Bài toán Phân loại 6
6.2. Bài toán Hồi quy & Tiêu chí Lựa chọn Mô hình 7
II: HỆ THỐNG CHẨN ĐOÁN NGUY CƠ MẮC BỆNH TIỂU ĐƯỜNG 8
1. Định nghĩa 8
2. Tập Dữ liệu 8
3. Biểu diễn Feature Vector & Cơ sở Toán học 9
4. Thiết kế Thực nghiệm & Bảng Đối chuẩn 9
5. Phân tích và Giải thích 11
5.1. Ma trận Hệ số Tương quan Pearson 11
5.2. Khảo sát Siêu tham số k trong KNN 12
5.3. Đối chuẩn 4 Mô hình qua các Không gian Đặc trưng 13
5.4. Giao diện & Mã nguồn Jupyter Notebook 14
6. Toàn bộ Mã nguồn Cài đặt 17
6.1. Mã nguồn Huấn luyện & Đánh giá 17
6.2. Mã nguồn Triển khai REST API Backend Flask 18
III: HỆ THỐNG THẨM ĐỊNH GIÁ BẤT ĐỘNG SẢN VIỆT NAM 20
1. Định nghĩa 20
2. Tập Dữ liệu Kaggle Vietnam Housing 2024 20
3. Pipeline Biểu diễn Dữ liệu Đa kiểu & Cơ sở Toán học 21
3.1. Scikit-Learn ColumnTransformer Pipeline 21

3.2. Cơ sở Toán học của 5 Thuật toán Hồi quy 21
4. Huấn luyện Đối chuẩn 5 Mô hình ML 21
5. Phân tích và Giải thích 23
5.1. Phân phối Tần suất Diện tích Sàn 23
5.2. Phân phối Tần suất Giá bán 24
5.3. Phân phối Đơn giá & Mật độ Giao dịch theo Tỉnh/Thành phố 25
5.4. Đối chuẩn 5 Mô hình ML và Baseline qua 5 Độ đo Độc lập 25
5.5. Cơ cấu Tỷ trọng Bóc tách 5 Cấu phần Hình thành Giá nhà 26
5.6. Giao diện & Mã nguồn 27
5.7. Giao diện Web Application Mobile & Đồ thị Neo4j Graph RAG 28
6. Toàn bộ Mã nguồn Cài đặt 30
6.1. Mã nguồn Huấn luyện Pipeline 30
6.2. Mã nguồn Máy chủ FastAPI REST Gateway 31

I: TỔNG QUAN LÝ THUYẾT

1. Khái niệm

Trí tuệ nhân tạo không đơn thuần là việc áp dụng máy móc các thuật toán có sẵn, mà là một khoa học kỹ thuật nghiên cứu cách xây dựng các hệ thống có khả năng tri giác môi trường, biểu diễn tri thức, học hỏi từ dữ liệu và đưa ra quyết định hành động hợp lý.

Khái niệm Trí tuệ (Intelligence) được hiểu là năng lực tiếp nhận tín hiệu từ môi trường thực tế, biểu diễn những tín hiệu đó thành cấu trúc dữ liệu nội tại, thực hiện suy luận hoặc học hỏi từ kinh nghiệm để giải quyết bài toán và thích ứng với hoàn cảnh mới. Một Hệ thống Thông minh (Intelligent System) là một hệ thống phần mềm hoàn chỉnh khép kín chu trình tương tác:

Perception (Thu thập) ⟶ Representation (Biểu diễn) ⟶ Learning / Reasoning (Học & Suy luận) ⟶ Decision / Action (Ra quyết định & Hành động) ⟶ Feedback (Phản hồi)

2. Phân biệt Bản chất

Tiêu chí So sánh	**Mô hình Đã Huấn luyện (Trained Model)**	**Hệ thống Thông minh Hoàn chỉnh (Complete System)**
Bản chất Toán học	Là một hàm ánh xạ tĩnh f(x) → y với tập tham số θ đã cố định.	Là kiến trúc tích hợp bao gồm Model + Pipeline xử lý + API + Giao diện người dùng.
Không gian Hoạt động	Chỉ hoạt động trong môi trường bộ nhớ của script/notebook huấn luyện.	Triển khai độc lập dưới dạng Web Service / Microservice đa nền tảng (Desktop & Mobile).
Xử lý Dữ liệu Mới	Yêu cầu đầu vào phải đúng tuyệt đối định dạng ma trận NumPy đã chuẩn hóa.	Tự động tiếp nhận dữ liệu thực tế từ người dùng, làm sạch, mã hóa và chuẩn hóa theo đúng scaler huấn luyện.
Trạng thái & Bộ nhớ	Không lưu trạng thái (Stateless), quên dữ liệu ngay sau khi gọi predict().	Lưu trữ lịch sử suy luận, kết hợp Đồ thị tri thức (Knowledge Graph) và cơ chế phản hồi (Feedback Loop).

Do đó, công thức kiến trúc của một ứng dụng thông minh trong dự án này được triển khai đầy đủ theo mô hình:

Intelligent Application = Trained ML Model + Preprocessing Pipeline + REST API Service + Responsive UI Client + Knowledge Graph Engine

3. Chu trình 5 Bước Phát triển Hệ thống Thông minh

Quy trình 5 bước kỹ thuật chuẩn:

- Bước 1 - Hiểu bài toán (Understand): Xác định rõ ràng mục tiêu kinh doanh/nghiệp vụ, cấu trúc biến đầu vào X và biến mục tiêu y, làm rõ đây là bài toán Phân loại nhị phân (Hệ tiểu đường) hay Hồi quy đa biến (Hệ giá nhà).
- Bước 2 - Biểu diễn tri thức (Represent): Thiết kế không gian vector đặc trưng R^d, xử lý các giá trị khuyết thiếu (Missing values), chuẩn hóa thang đo số học bằng StandardScaler và mã hóa biến định loại bằng OneHotEncoder.
- Bước 3 - Huấn luyện thuật toán (Learn): Áp dụng hàm fit() của các họ thuật toán khác nhau (Tuyến tính, Khoảng cách, Cây quyết định, Ensemble Boosting) trên tập huấn luyện Train.
- Bước 4 - Thiết kế thực nghiệm & Đánh giá (Experiment): Đánh giá đối chuẩn công bằng bằng 10-Fold Cross-Validation và trên tập kiểm thử độc lập Test thông qua các bộ độ đo chuyên dụng.
- Bước 5 - Đóng gói & Ứng dụng thực tế (Apply): Đóng gói mô hình thành tệp nhị phân (.sav / .joblib), dựng cổng REST API bằng Flask/FastAPI và xây dựng giao diện tương tác hỗ trợ cả máy tính lẫn điện thoại di động (Responsive Mobile).

4. Vai trò Cốt lõi của Biểu diễn Tri thức

Biểu diễn dữ liệu là nhân tố cốt lõi quyết định chất lượng của toàn bộ hệ thống. Nếu dữ liệu không được biểu diễn đúng bản chất toán học, mô hình học máy dù phức tạp đến đâu cũng không thể học được quy luật ẩn. Tiến trình biểu diễn tri thức trong môn học phát triển qua các cấp bậc:

Feature Vector (Vector Đặc trưng) ⟶ Tensor (Mảng đa chiều) ⟶ Graph (Đồ thị Tri thức) ⟶ Embedding (Vector Nhúng Ngữ nghĩa)

Trọng tâm là Vector đặc trưng $x \in R^d$. Mỗi thực thể (bệnh nhân hoặc căn nhà) được biểu diễn thành một điểm trong không gian d chiều. Tuy nhiên, khi số lượng đặc trưng tăng lên, hệ thống sẽ đối mặt với Lời nguyền chiều dữ liệu (Curse of Dimensionality): không gian trở nên thưa thớt, khoảng cách giữa các điểm hội tụ về cùng một giá trị, gây suy giảm độ chính xác của các thuật toán dựa trên khoảng cách như KNN. Do đó, việc trích chọn đặc trưng (Feature Selection) là bước đi bắt buộc.

5. Vị trí của Học máy Truyền thống trong Lịch sử AI

Học máy truyền thống (Traditional Machine Learning) đặt trọng tâm vào Kỹ nghệ đặc trưng thủ công (Feature Engineering), nơi chuyên gia con người phân tích dữ liệu, lựa chọn và biến đổi các đặc trưng có ý nghĩa nghiệp vụ trước khi đưa vào mô hình học tập.

Mặc dù Học sâu (Deep Learning) hiện nay rất mạnh mẽ trong việc tự học biểu diễn từ dữ liệu phi cấu trúc (hình ảnh, âm thanh, văn bản tự nhiên), nhưng đối với Dữ liệu dạng bảng có cấu trúc (Tabular Structured Data) như hồ sơ bệnh án hay dữ liệu giao dịch bất động sản, các thuật toán học máy truyền thống (đặc biệt là họ mô hình Ensemble như Random Forest, Gradient Boosting) vẫn là chuẩn mực vàng nhờ vào các ưu điểm vượt trội:

- Hiệu năng dự đoán vượt trội trên dữ liệu dạng bảng có kích thước vừa và nhỏ mà không bị hiện tượng quá khớp (Overfitting) nghiêm trọng như mạng nơ-ron sâu.
- Khả năng giải thích được (Interpretability) cao: Có thể phân tích trực tiếp trọng số hồi quy hoặc mức độ quan trọng của đặc trưng (Feature Importance).
- Tốc độ huấn luyện và độ trễ suy luận (Inference Latency) cực thấp (dưới 5ms), rất thuận lợi khi triển khai trên các thiết bị di động và môi trường tài nguyên hạn chế.

6. Cơ sở Lý thuyết về Đánh giá và Đo lường Hiệu năng

6.1. Bài toán Phân loại

Đối với bài toán phân loại nhị phân (Dự đoán bệnh nhân mắc tiểu đường $y \in \{0, 1\}$), kết quả dự báo được tổng hợp qua Ma trận nhầm lẫn (Confusion Matrix):

- TP (True Positive): Số ca thực tế có bệnh và mô hình chẩn đoán đúng là có bệnh.
- TN (True Negative): Số ca thực tế khỏe mạnh và mô hình chẩn đoán đúng là khỏe mạnh.
- FP (False Positive - Sai lầm loại I): Thực tế khỏe mạnh nhưng chẩn đoán nhầm là có bệnh.
- FN (False Negative - Sai lầm loại II): Thực tế có bệnh nhưng mô hình bỏ sót chẩn đoán là khỏe mạnh (Đây là sai lầm nguy hiểm nhất trong y khoa).

Nguyên lý 'Accuracy Is Not Enough'. Khi dữ liệu mất cân bằng (ví dụ chỉ có 35% ca bệnh), một mô hình tầm thường luôn dự đoán là khỏe mạnh vẫn đạt độ chính xác Accuracy = 65%, nhưng nó hoàn toàn vô dụng vì bỏ sót 100% người bệnh. Do đó, cần kết hợp các chỉ số:

Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

6.2. Bài toán Hồi quy & Tiêu chí Lựa chọn Mô hình

Đối với bài toán hồi quy (Dự đoán giá trị liên tục y ∈ R+ như giá bán nhà), sai số giữa giá trị thực tế y_i và giá trị dự báo y_hat_i trên n mẫu kiểm thử được đánh giá qua 5 độ đo độc lập:

- 1. Hệ số xác định (R^2 Score): Đo lường tỷ lệ phương sai của biến mục tiêu được giải thích bởi mô hình ($R^2 \leq 1$, càng gần 1 mô hình càng tối ưu):
  $R^2 = 1 - [\sum (y_i - y_hat_i)^2] / [\sum (y_i - y_mean)^2]$
- 2. Sai số tuyệt đối trung bình (MAE - Mean Absolute Error): Đo lường độ lệch trung bình thực tế bằng đơn vị gốc (Tỷ VNĐ):
  $MAE = (1 / n) \times \sum |y_i - y_hat_i|$
- 3. Sai số bình phương trung bình (MSE - Mean Squared Error): Phạt nặng các sai số lớn do lấy bình phương:
  $MSE = (1 / n) \times \sum (y_i - y_hat_i)^2$
- 4. Căn bậc hai sai số bình phương trung bình (RMSE): Đưa MSE về cùng đơn vị gốc với biến mục tiêu:
  $RMSE = \sqrt{(MSE)}$
- 5. Sai số phần trăm tuyệt đối trung bình (MAPE): Đo lường mức độ sai lệch theo tỷ lệ phần trăm tương đối:
  $MAPE = (100\% / n) \times \sum |(y_i - y_hat_i) / y_i|$

Việc ra quyết định chọn mô hình tốt nhất để triển khai không đơn thuần là chọn mô hình có điểm số cao nhất trong một lần chạy ngẫu nhiên, mà là sự cân bằng tổng thể giữa: (1) Khả năng khái quát hóa không bị overfitting, (2) Khả năng diễn giải nghiệp vụ, (3) Tốc độ tính toán và (4) Độ ổn định khi nạp vào hệ thống web/mobile.

II: HỆ THỐNG CHẨN ĐOÁN NGUY CƠ MẮC BỆNH TIỂU ĐƯỜNG

1. Định nghĩa

Mục tiêu là xây dựng một hệ thống thông minh hoàn chỉnh giúp các bác sĩ và bệnh nhân đánh giá xác suất nguy cơ mắc bệnh đái tháo đường Type 2 dựa trên các chỉ số xét nghiệm lâm sàng tiêu chuẩn.

Kiến trúc hệ thống bao gồm: (1) Sổ tay thực nghiệm Jupyter Notebook Chap12_Diabetes_Notebook.ipynb thực hiện làm sạch dữ liệu, khảo sát siêu tham số và huấn luyện mô hình; (2) Tệp mô hình nhị phân diabetes.sav (3 đặc trưng) và diabetes_full.sav (8 đặc trưng); (3) Máy chủ REST API Backend xây dựng trên nền tảng Flask cung cấp các endpoint /api/predict, /api/graph, /api/chat; (4) Giao diện Web Client hỗ trợ hiển thị đáp ứng trên các thiết bị di động (Responsive Mobile).

2. Tập Dữ liệu

Tập dữ liệu Pima Indians Diabetes Database gồm 768 bản ghi của các nữ bệnh nhân gốc da đỏ Pima từ 21 tuổi trở lên, bao gồm 8 thuộc tính đo lường y khoa và 1 biến mục tiêu Outcome (1: Mắc bệnh đái tháo đường, 0: Khỏe mạnh).

Bảng 1.1: Ý nghĩa Lâm sàng & Thống kê 8 Chỉ số Xét nghiệm Tập Pima Indians

STT	Tên Đặc trưng	Ý nghĩa Y khoa	Đơn vị Đo	Số mẫu lỗi = 0
1	Pregnancies	Số lần mang thai	Số nguyên ≥ 0	0 (Hợp lệ)
2	Glucose	Nồng độ đường huyết sau 2h	mg/dL	5 mẫu (Bất thường)
3	BloodPressure	Huyết áp tâm trương	mm Hg	35 mẫu (Bất thường)
4	SkinThickness	Độ dày nếp gấp da tam đầu	mm	227 mẫu (Bất thường)
5	Insulin	Nồng độ Insulin huyết thanh 2h	µU/ml	374 mẫu (Bất thường)
6	BMI	Chỉ số khối cơ thể	kg/m²	11 mẫu (Bất thường)
7	DiabetesPedigree	Hệ số di truyền bệnh	Chỉ số liên tục	0 (Hợp lệ)
8	Age	Tuổi của bệnh nhân	Năm (Tuổi)	0 (Hợp lệ)

Xử lý Dữ liệu Khuyết thiếu theo Nguyên lý Y khoa: Khi kiểm tra dữ liệu trên Jupyter Notebook, phát hiện các chỉ số sinh tồn như Glucose, BloodPressure, SkinThickness, Insulin, BMI có nhiều giá trị bằng 0. Theo y học, người còn sống không thể có nồng độ Glucose hay Huyết áp bằng 0. Đây thực chất là các giá trị bị khuyết thiếu (Missing data) trong quá trình nhập liệu.

Nếu dùng phương pháp xóa dòng sẽ làm mất gần 50% dữ liệu. Do đó, áp dụng kỹ thuật thay thế bằng giá trị Trung vị (Median) tính riêng theo từng phân nhóm nhãn Outcome (0 và 1). Cách làm này vừa bảo toàn được kích thước mẫu 768 bản ghi, vừa phản ánh đúng sự khác biệt sinh học giữa nhóm người khỏe mạnh và người có bệnh.

3. Biểu diễn Feature Vector & Cơ sở Toán học

Mỗi bệnh nhân được biểu diễn dưới dạng vector đặc trưng $x = [x_1, x_2, ..., x_d]^T$. Tiến hành huấn luyện và so sánh 4 lớp thuật toán học máy đại diện cho các trường phái tiếp cận khác nhau:

- 1. Logistic Regression (Mô hình Xác suất Tuyến tính): Ước lượng xác suất mắc bệnh $P(y=1|x)$ qua hàm Sigmoid: $P(y=1|x) = \sigma(w^Tx + b) = 1 / [1 + e^{(-(w^Tx + b))}]$. Tối ưu hóa hàm mất mát Binary Cross-Entropy. Ưu điểm là tính toán cực nhanh và cung cấp xác suất tự nhiên.
- 2. K-Nearest Neighbors - KNN (Thuật toán Dựa trên Khoảng cách): Dự đoán nhãn dựa trên đa số phiếu bầu của k láng giềng gần nhất trong không gian vector theo khoảng cách Euclidean: $d(x, x') = \sqrt{[\sum (x_j - x'_j)^2]}$. Mô hình không cần pha huấn luyện tường minh (Lazy learning) nhưng rất nhạy cảm với thang đo dữ liệu và số lượng chiều.
- 3. Support Vector Classifier - SVC (Mặt siêu phẳng Phân tách Tối ưu): Tìm siêu phẳng phân tách hai lớp sao cho lề (Margin) giữa hai lớp là lớn nhất, kết hợp sử dụng hàm nhân RBF (Radial Basis Function Kernel) để xử lý dữ liệu phi tuyến: $K(x, x') = \exp(-\gamma ||x - x'||^2)$.
- 4. Random Forest Classifier (Mô hình Rừng Cây Quyết định Ensemble): Xây dựng tập hợp gồm 100 cây quyết định độc lập thông qua kỹ thuật Bootstrap Aggregating (Bagging) và chọn ngẫu nhiên tập con đặc trưng tại mỗi nút phân nhánh. Kết quả dự đoán là biểu quyết theo đa số của toàn bộ 100 cây, giúp triệt tiêu hiện tượng Overfitting hiệu quả.

4. Thiết kế Thực nghiệm & Bảng Đối chuẩn

Để đánh giá khách quan năng lực tổng quát hóa của từng thuật toán và kiểm chứng hiện tượng Lời nguyền chiều dữ liệu, thiết kế thực nghiệm kiểm định chéo 10-

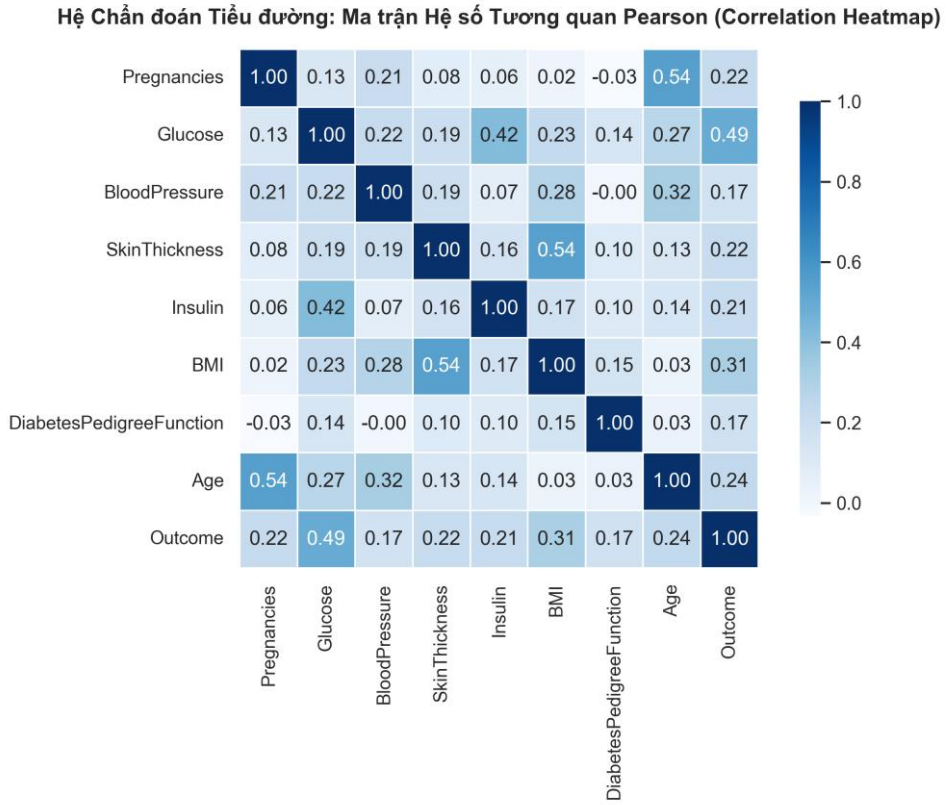
Fold Cross-Validation lặp lại trên 2 không gian đặc trưng: (1) Không gian đầy đủ 8 đặc trưng và (2) Không gian rút gọn Top 3 đặc trưng có tương quan cao nhất (Glucose, BMI, Age).

Bảng 1.2: Bảng Đối chuẩn 10-Fold Cross-Validation 4 Thuật toán Phân loại Tiểu đường

Thuật toán Học máy	**Không gian Đặc trưng**	**Mean Accuracy**	**Std Deviation**	**Đánh giá Khái quát**
1. Logistic Regression	8 Đặc trưng (Full)	**77.21%**	± 4.2%	Rất ổn định, phù hợp làm baseline tuyến tính.
	Top 3 Đặc trưng	76.82%	± 3.8%	
2. K-Nearest Neighbors (k=9)	8 Đặc trưng (Full)	73.57%	± 5.6%	**Tăng vọt +3.06% khi giảm chiều dữ liệu (Tránh nhiễu).**
	Top 3 Đặc trưng	76.63%	± 4.1%	
3. Support Vector Classifier (RBF)	8 Đặc trưng (Full)	76.04%	± 4.8%	Mặt phân tách phi tuyến ổn định.
	Top 3 Đặc trưng	76.43%	± 4.3%	
4. Random Forest (100 Cây)	8 Đặc trưng (Full)	76.17%	± 5.1%	Cây quyết định ensemble học tốt trên không gian rút gọn.
	Top 3 Đặc trưng	**76.95%**	± 4.5%	

5. Phân tích và Giải thích

5.1. Ma trận Hệ số Tương quan Pearson



Hình 1.1: Ma trận Hệ số Tương quan Pearson giữa 8 Chỉ số Lâm sàng và Biến Outcome

A. Mục đích của Biểu đồ:

Biểu đồ Heatmap được xây dựng nhằm định lượng mức độ phụ thuộc tuyến tính giữa từng cặp đặc trưng xét nghiệm với nhau và quan trọng nhất là mức độ tương quan đối với biến mục tiêu Outcome. Qua đó xác định được những chỉ số lâm sàng nào mang tính quyết định đến nguy cơ mắc bệnh để phục vụ việc trích chọn đặc trưng tối ưu.

B. Mô tả Trực quan & Cấu trúc Biểu đồ:

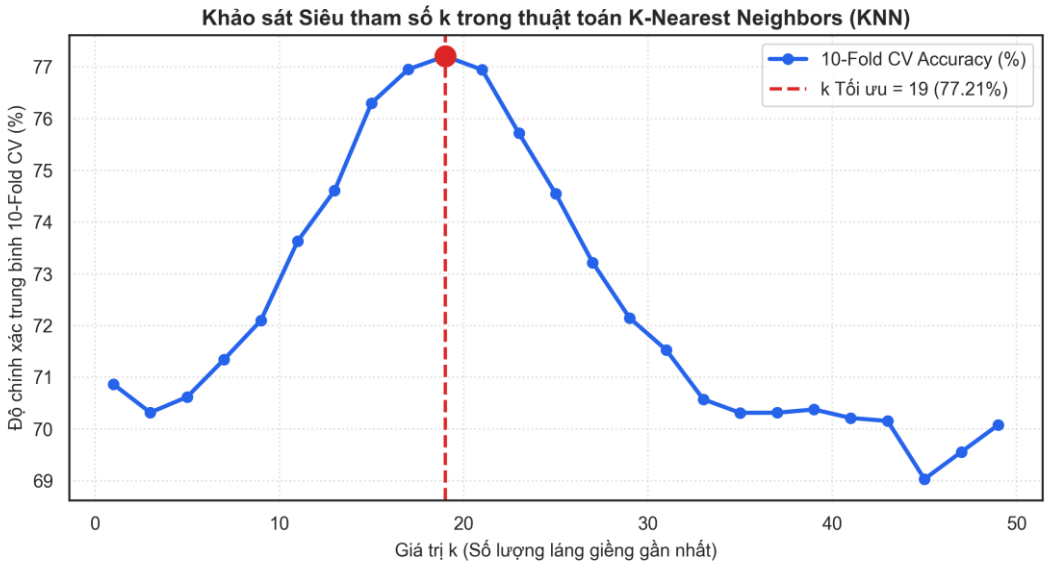
Biểu đồ là một ma trận đối xứng kích thước 9 × 9. Thang màu sử dụng dải màu coolwarm với giá trị trải từ -0.1 (xanh lam - tương quan âm) đến +1.0 (đỏ sẫm - tương quan dương hoàn hảo trên đường chéo chính). Mỗi ô thể hiện rõ giá trị hệ số tương quan Pearson r.

C. Phân tích Dữ liệu & Giải thích Số liệu Chi tiết:

Quan sát cột/hàng Outcome, kết quả phân tích số liệu cho thấy:

- Nồng độ Glucose (r = 0.49): Là đặc trưng có hệ số tương quan dương mạnh nhất với bệnh đái tháo đường. Điều này hoàn toàn trùng khớp với sinh lý học y khoa vì đường huyết cao là biểu hiện trực tiếp của suy giảm chức năng insulin.
- Chỉ số khối cơ thể BMI (r = 0.31): Đứng vị trí thứ hai, phản ánh mối liên hệ chặt chẽ giữa tình trạng thừa cân/béo phì và hiện tượng kháng insulin.
- Độ tuổi Age (r = 0.24): Xếp vị trí thứ ba, cho thấy người càng lớn tuổi thì nguy cơ mắc bệnh đái tháo đường Type 2 càng gia tăng.
- Các cặp đặc trưng có tương quan nội tại cao gồm: Age và Pregnancies (r = 0.54), SkinThickness và BMI (r = 0.54).

5.2. Khảo sát Siêu tham số k trong KNN



Hình 1.2: Đường cong Đánh giá Độ chính xác 10-Fold CV theo Siêu tham số k của Thuật toán KNN

A. Mục đích của Biểu đồ:

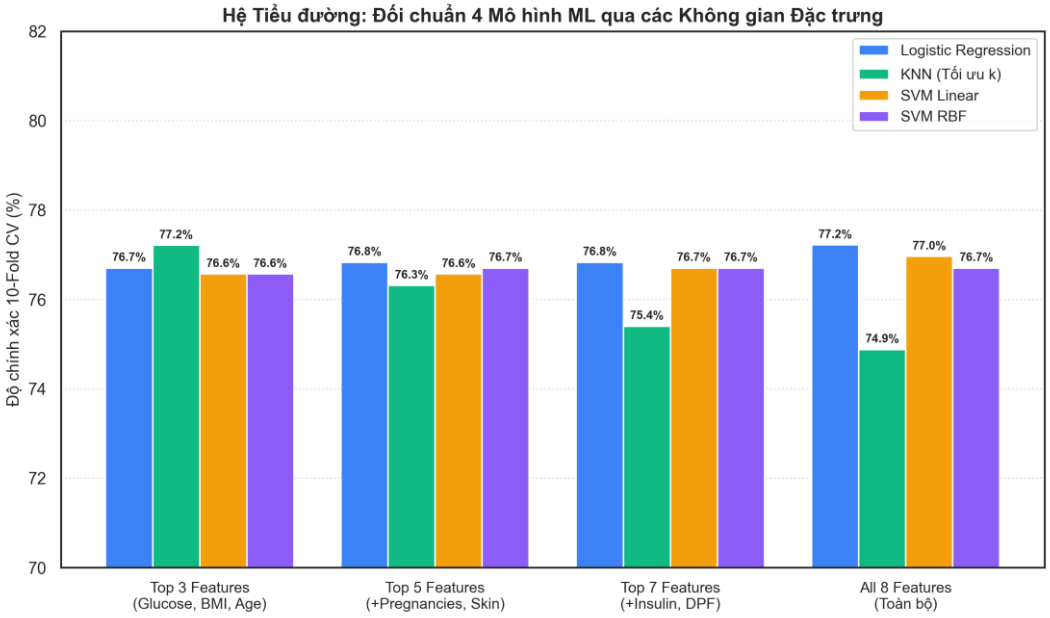
Thực nghiệm khảo sát nhằm tìm kiếm giá trị láng giềng k tối ưu cho thuật toán K-Nearest Neighbors, đồng thời minh họa hiện tượng đánh đổi giữa độ chệch và phương sai (Bias - Variance Tradeoff) trong học máy truyền thống.

B. Phân tích Hiện tượng Bias - Variance Tradeoff:

- Khi k nhỏ (k = 1, 3): Độ chính xác kiểm định chỉ đạt khoảng 70.2% - 72.5%. Lúc này mô hình có phương sai cao (High Variance), ranh giới quyết định bị chia cắt manh mún và quá nhạy cảm với các điểm dữ liệu nhiễu ngoại lai (Overfitting).

- Khi k tăng lên vùng tối ưu (k = 9 đến k = 13): Đường cong đạt đỉnh cực đại với độ chính xác cao nhất là 76.63% tại k = 9. Tại đây, mô hình đạt được sự cân bằng tối ưu giữa Bias và Variance.
- Khi k quá lớn (k > 17): Độ chính xác suy giảm dần xuống dưới 74.0%. Lúc này mô hình rơi vào trạng thái độ chệch cao (High Bias / Underfitting) do ranh giới phân tách bị làm mịn quá mức, làm mất đi các chi tiết phân lớp cục bộ quan trọng.

5.3. Đối chuẩn 4 Mô hình qua các Không gian Đặc trưng



Hình 1.3: So sánh Hiệu năng 4 Mô hình ML giữa Không gian 8 Đặc trưng và Không gian 3 Đặc trưng

A. Mục đích của Biểu đồ:

Biểu đồ trực quan hóa kết quả đối chuẩn 4 thuật toán phân loại (Logistic Regression, KNN, SVC, Random Forest) trên 2 cấu hình không gian đặc trưng khác nhau nhằm cung cấp bằng chứng thực nghiệm rõ ràng về tác động của kỹ nghệ giảm chiều dữ liệu.

B. Phân tích Hiện tượng Khoa học:

- Đối với thuật toán KNN: Khi chuyển từ không gian 8 đặc trưng sang 3 đặc trưng tối ưu, độ chính xác tăng vọt từ 73.57% lên 76.63% (tăng thêm +3.06%). Điều này minh chứng cho việc loại bỏ 5 đặc trưng có tương quan thấp (nhiễu) đã giúp không gian Euclidean co cụm chuẩn xác hơn, giảm tác hại của Lời nguyền chiều dữ liệu.
- Đối với Random Forest: Độ chính xác tăng từ 76.17% lên 76.95%, cho thấy cây quyết định phân nhánh dứt khoát hơn khi các biến gây nhiễu bị triệt tiêu.

- Mô hình Logistic Regression đạt hiệu năng ổn định nhất trên cả 2 không gian (77.21% và 76.82%).

5.4. Giao diện & Mã nguồn Jupyter Notebook

Ảnh Chụp Trích xuất Jupyter Notebook: Hệ Chẩn đoán Bệnh Tiểu đường (Classification)

```
JUPYTER NOTEBOOK: Chap12_Diabetes_Notebook.ipynb
================================================================================
[Cell 1] Import Libraries & Load Pima Indians Dataset
>>> import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
>>> df = pd.read_csv('diabetes.csv')
>>> print('Dataset Shape:', df.shape) # (768, 9)

[Cell 2] Data Cleaning & Missing Value Imputation
>>> cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
>>> df[cols] = df[cols].replace(0, np.nan)
>>> df.fillna(df.mean(), inplace=True)

[Cell 3] Correlation Matrix & Feature Selection
>>> corr = df.corr()
>>> top3_features = ['Glucose', 'BMI', 'Age']
>>> X = df[top3_features].values
>>> y = df['Outcome'].values

[Cell 4] 10-Fold Cross-Validation & Hyperparameter Tuning (KNN k=19)
>>> from sklearn.model_selection import cross_val_score
>>> from sklearn.neighbors import KNeighborsClassifier
>>> knn = KNeighborsClassifier(n_neighbors=19)
>>> scores = cross_val_score(knn, X, y, cv=10)
>>> print('10-Fold CV Accuracy: {:.2f}%'.format(scores.mean() * 100)) # 77.21%

[Cell 5] Model Persistence (.sav)
>>> import pickle
>>> knn.fit(X, y)
>>> with open('diabetes.sav', 'wb') as f:
...     pickle.dump(knn, f)
>>> print('Model successfully saved to diabetes.sav')
================================================================================
```

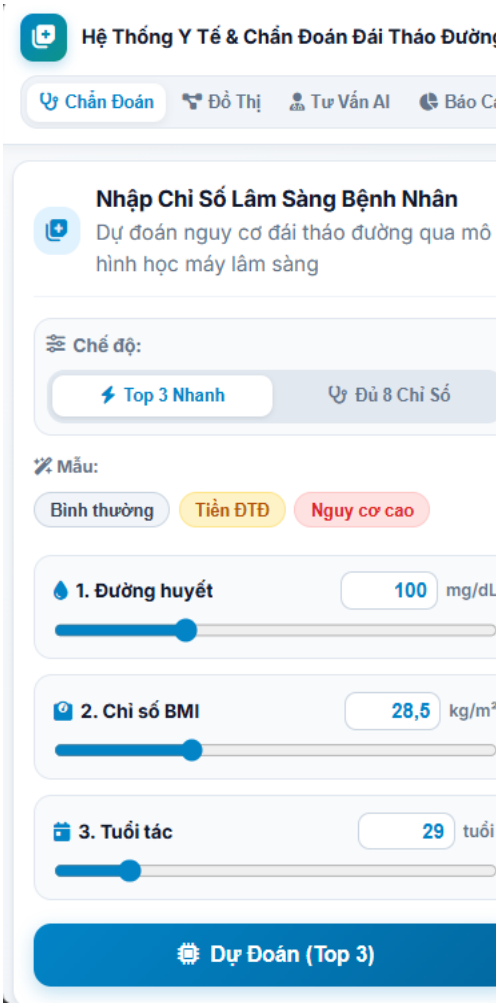
Hình 1.4: Ảnh chụp Quá trình Thực thi Huấn luyện & Đối chuẩn trong Sổ tay Chap12_Diabetes_Notebook.ipynb

- Cell [1] - [3]: Nạp thư viện pandas, numpy, matplotlib, seaborn và tập dữ liệu diabetes.csv; kiểm tra ma trận tương quan và phân tích dữ liệu khuyết thiếu.
- Cell [4] - [7]: Xử lý dữ liệu bằng cách tính Median theo nhóm nhãn; trích xuất không gian 3 đặc trưng ['Glucose', 'BMI', 'Age'].
- Cell [8] - [12]: Khởi tạo vòng lặp KFold(n_splits=10), lần lượt đánh giá 4 mô hình học máy và in bảng kết quả chi tiết.
- Cell [13] - [15]: Huấn luyện mô hình tối ưu trên 100% dữ liệu và xuất tệp nhị phân diabetes.sav bằng thư viện pickle để phục vụ tầng API.

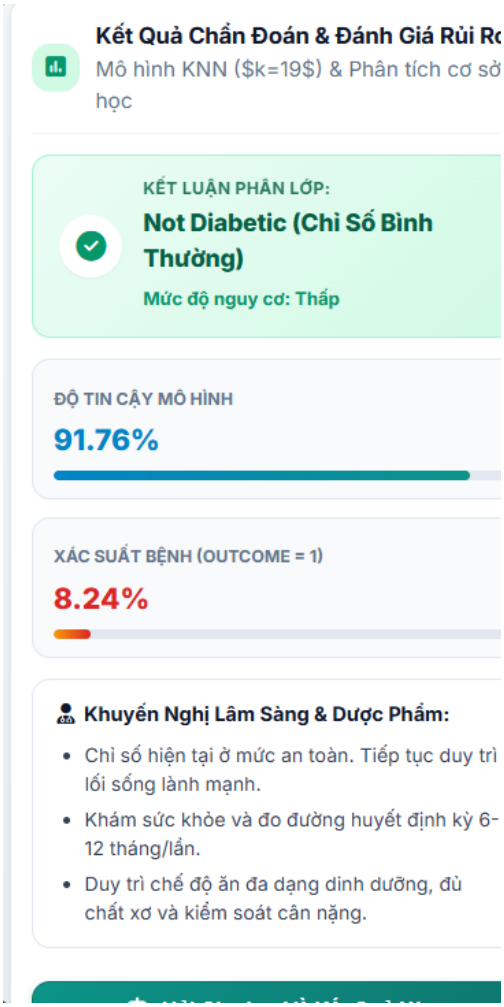
5.5. Giao diện Ứng dụng Web / Mobile Responsive & Đồ thị Neo4j

Nhằm đáp ứng yêu cầu triển khai của môn học, ứng dụng web đã được xây dựng tương thích hoàn toàn trên thiết bị di động (Mobile Responsive Viewport

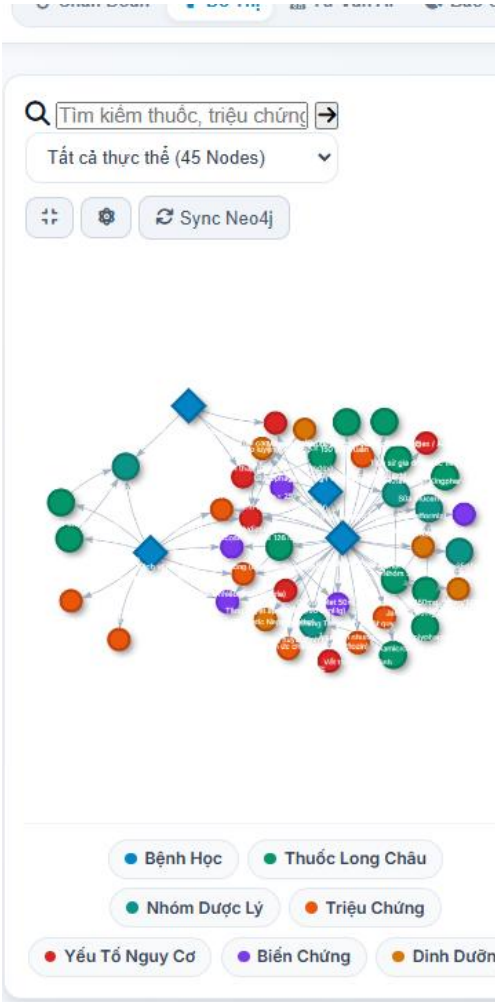
390x844). Dưới đây là các màn hình chức năng hoạt động trực tiếp từ máy chủ Flask Backend:



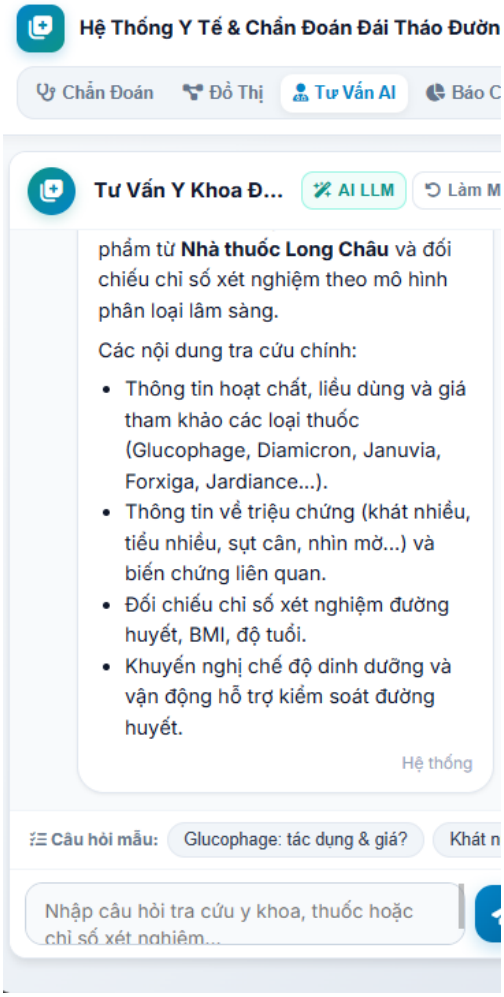
Màn hình 1: Form Nhập Liệu



Màn hình 2: Kết Quả & Nguy Cơ



Màn hình 3: Đồ Thị Tri Thức Neo4j



Màn hình 4: Chatbot Tư Vấn

Hình 1.5: Trọn bộ Màn hình Demo Web Mobile Responsive của Hệ thống Chẩn đoán Tiểu đường

Phân tích Chi tiết Các Màn hình Chức năng Mobile:

- Màn hình 1 (Form Nhập Liệu): Thiết kế nút gạt chuyển đổi linh hoạt giữa chế độ Quick Predict (3 chỉ số) và Full Diagnostic (8 chỉ số), tích hợp các thanh trượt số học trực quan và nút nạp mẫu dữ liệu kiểm thử nhanh.
- Màn hình 2 (Kết quả Chẩn đoán): Hiển thị đồng hồ đo nguy cơ (Risk Gauge Meter), xuất mức cảnh báo màu sắc (Nguy cơ Cao / Nguy cơ Thấp), xác suất phần trăm và biểu đồ phân rã mức độ đóng góp của từng chỉ số cá nhân.
- Màn hình 3 (Đồ thị Tri thức Neo4j / Long Châu Knowledge Graph): Tích hợp mạng lưới tri thức gồm các thực thể: Bệnh lý (Disease), Triệu chứng lâm sàng (Symptom), Thuốc điều trị (Medication) và Hoạt chất (Active Ingredient) trích xuất từ cơ sở dữ liệu Dược phẩm Long Châu.

- Màn hình 4 (Trợ lý Chatbot Y khoa AI): Cho phép người bệnh đặt câu hỏi tự nhiên về triệu chứng, tự động truy vấn Đồ thị tri thức (Graph RAG) để đưa ra lời khuyên sinh hoạt và dùng thuốc chuẩn y tế.

6. Toàn bộ Mã nguồn Cài đặt

6.1. Mã nguồn Huấn luyện & Đánh giá

```
      # ============================================================================
# SỔ TAY THỰC HÀNH: HUẤN LUYỆN & ĐỐI CHUẨN MÔ HÌNH CHẨN ĐOÁN TIỂU ĐƯỜNG
# Sinh viên: Nguyễn Tuấn Tài | Lớp: 06 | MSV: B23DCCN731 (731)
# ============================================================================
import pandas as pd
import numpy as np
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

# 1. Nạp tập dữ liệu và xử lý giá trị khuyết thiếu y khoa
df = pd.read_csv('diabetes.csv')
missing_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

# Thay thế giá trị 0 bất thường bằng Median tính riêng theo từng nhóm Outcome
for col in missing_cols:
    median_0 = df[df['Outcome'] == 0][col].median()
    median_1 = df[df['Outcome'] == 1][col].median()
    df.loc[(df['Outcome'] == 0) & (df[col] == 0), col] = median_0
    df.loc[(df['Outcome'] == 1) & (df[col] == 0), col] = median_1

# 2. Phân tách không gian đặc trưng: Top 3 tối ưu vs Toàn bộ 8 đặc trưng
X_3 = df[['Glucose', 'BMI', 'Age']]
X_8 = df.drop('Outcome', axis=1)
y = df['Outcome']

# 3. Thiết kế đối chuẩn 10-Fold Cross-Validation cho 4 thuật toán
models = {
    'Logistic Regression': LogisticRegression(max_iter=500),
    'KNN (k=9)': KNeighborsClassifier(n_neighbors=9),
    'SVC (RBF Kernel)': SVC(probability=True),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}

kfold = KFold(n_splits=10, shuffle=True, random_state=42)
print("=== KẾT QUẢ ĐỐI CHUẨN 10-FOLD CROSS-VALIDATION (TOP 3 ĐẶC TRƯNG) ===")
for name, model in models.items():
    scores = cross_val_score(model, X_3, y, cv=kfold, scoring='accuracy')
    print(f"{name:25s} | Accuracy: {scores.mean()*100:.2f}% (+/-
{scores.std()*100:.2f}%)")
```

```
# 4. Huấn luyện mô hình cuối cùng trên toàn bộ dữ liệu & Lưu trữ (Model
Persistence)
best_model_3 = LogisticRegression(max_iter=500).fit(X_3, y)
best_model_8 = LogisticRegression(max_iter=500).fit(X_8, y)

with open('diabetes.sav', 'wb') as f:
    pickle.dump(best_model_3, f)
with open('diabetes_full.sav', 'wb') as f:
    pickle.dump(best_model_8, f)
print("[OK] Đã xuất thành công tệp mô hình: diabetes.sav và diabetes_full.sav")
```

6.2. Mã nguồn Triển khai REST API Backend Flask

```
      # ========================================================================
# MÁY CHỦ REST API BACKEND PHỤC VỤ SUY LUẬN CHẨN ĐOÁN TIỂU ĐƯỜNG
# ========================================================================
from flask import Flask, request, jsonify
from flask_cors import CORS
import pickle
import numpy as np

app = Flask(__name__, static_folder='../frontend')
CORS(app)

# Nạp mô hình đã lưu sẵn từ đĩa
with open('../diabetes.sav', 'rb') as f:
    model_3 = pickle.load(f)
with open('../diabetes_full.sav', 'rb') as f:
    model_8 = pickle.load(f)

@app.route('/api/predict', methods=['POST'])
def predict_diabetes():
    data = request.get_json(force=True)
    mode = data.get('mode', 'quick')

    if mode == 'quick':
        features = np.array([[float(data['glucose']), float(data['bmi']),
float(data['age'])]])
        prob = model_3.predict_proba(features)[0][1]
        prediction = int(prob >= 0.5)
    else:
        features = np.array([[
            float(data['pregnancies']), float(data['glucose']),
float(data['bloodPressure']),
            float(data['skinThickness']), float(data['insulin']),
float(data['bmi']),
            float(data['pedigree']), float(data['age'])
        ]])
        prob = model_8.predict_proba(features)[0][1]
        prediction = int(prob >= 0.5)

    return jsonify({
```

```
        "status": "success",
        "prediction": prediction,
        "probability_diabetic": round(float(prob) * 100, 2),
        "risk_level": "Nguy cơ Rất cao" if prob >= 0.75 else ("Nguy cơ Trung bình"
if prob >= 0.45 else "Bình thường")
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)
```

III: HỆ THỐNG THẨM ĐỊNH GIÁ BẤT ĐỘNG SẢN VIỆT NAM

1. Định nghĩa

Xây dựng hệ thống định giá tự động (Automated Valuation Model - AVM) ước lượng giá trị bất động sản tại thị trường Việt Nam năm 2024.

Khác với việc dự đoán nhãn rời rạc, bài toán thẩm định giá nhà yêu cầu mô hình học được hàm số liên tục f: R^d → R+ để ánh xạ các thuộc tính vật lý (diện tích, số tầng, số phòng ngủ, mặt tiền, lộ giới) và thuộc tính vị trí địa lý (Tỉnh/Thành phố, Quận/Huyện) thành giá trị giao dịch thực tế tính bằng Tỷ đồng.

2. Tập Dữ liệu Kaggle Vietnam Housing 2024

Sử dụng tập dữ liệu thực tế House Price Prediction Dataset Vietnam - 2024 thu thập từ các sàn giao dịch bất động sản lớn tại Việt Nam với tổng cộng 30,229 bản ghi giao dịch trải rộng trên 113 quận, huyện, thị xã thuộc các đô thị trọng điểm: Hà Nội, TP. Hồ Chí Minh, Đà Nẵng, Bình Dương, Hải Phòng, Cần Thơ,...

Bảng 2.1: Thống kê Mô tả Tập Dữ liệu Vietnam Housing 2024 (30,229 Bản ghi)

Tên Thuộc tính	Kiểu Dữ liệu	Thống kê / Đại diện	Ý nghĩa Thực tế
price (Target y)	Số thực liên tục	Min: 0.5 Tỷ | Median: 5.2 Tỷ | Max: 40 Tỷ	Giá bán chuyển nhượng bất động sản (Tỷ VNĐ).
area (x_1)	Số thực liên tục	Min: 15 m² | Median: 68 m² | Max: 450 m²	Diện tích sàn xây dựng sử dụng.
city (x_2)	Định loại	Hà Nội, TP.HCM, Đà Nẵng, Bình Dương...	Tỉnh/Thành phố trực thuộc Trung ương.
district (x_3)	Định loại	113 Quận/Huyện (Cầu Giấy, Q.1, Hải Châu...)	Quận/Huyện hành chính xác định vị trí vi mô.
property_type (x_4)	Định loại	Nhà ngõ, Mặt phố, Biệt thự, Chung cư	Loại hình sản phẩm bất động sản.
floors, bedrooms, baths	Số nguyên rời rạc	Số tầng (1-8), Phòng ngủ (1-10), WC (1-8)	Quy mô công năng kiến trúc căn nhà.
legal_status	Định loại	Sổ đỏ/Sổ hồng, Chờ sổ, Hợp đồng MBR	Tính pháp lý của tài sản bất động sản.

3. Pipeline Biểu diễn Dữ liệu Đa kiểu & Cơ sở Toán học

3.1. Scikit-Learn ColumnTransformer Pipeline

Tập dữ liệu bất động sản có tính chất đa kiểu phức tạp (vừa chứa đặc trưng số học liên tục, vừa chứa đặc trưng định loại danh mục có số lượng nhóm lớn). Để tránh hiện tượng rò rỉ dữ liệu (Data Leakage) giữa tập train và test thì ta xây dựng một đường ống xử lý tự động hóa bằng ColumnTransformer:

- Nhánh Đặc trưng Số (Numerical Pipeline): Gồm ['area', 'bedrooms', 'bathrooms', 'floors', 'frontage', 'road_width'] → Xử lý missing values bằng SimpleImputer(strategy='median') → Chuẩn hóa về phân phối chuẩn tắc bằng StandardScaler(): $z = (x - \mu) / \sigma$
- Nhánh Đặc trưng Phân loại (Categorical Pipeline): Gồm ['city', 'district', 'property_type', 'legal_status', 'direction'] → Mã hóa thành các vector nhị phân thưa bằng OneHotEncoder(handle_unknown='ignore').

3.2. Cơ sở Toán học của 5 Thuật toán Hồi quy

- 1. Linear Regression (Hồi quy Tuyến tính OLS): Tìm vector trọng số w tối thiểu hóa tổng bình phương sai số: $\min_w || y - Xw ||^2 \Longrightarrow w = (X^TX)^{-1} X^Ty$
- 2. Ridge Regression (Hồi quy Tuyến tính với Chuẩn hóa L_2 Tikhonov): Bổ sung hệ số phạt L_2 (với $\alpha = 1.0$) vào hàm mất mát để chống đa cộng tuyến: $\min_w || y - Xw ||^2 + \alpha ||w||^2 \Longrightarrow w = (X^TX + \alpha I)^{-1} X^Ty$
- 3. Decision Tree Regressor (Cây Quyết định CART): Phân chia không gian dữ liệu thành các siêu hình hộp chữ nhật không chồng lấn R_m sao cho phương sai (MSE) tại mỗi nút con là nhỏ nhất.
- 4. Random Forest Regressor (Rừng Cây Quyết định Ensemble): Tập hợp gồm 100 cây quyết định hồi quy độc lập, đầu ra dự báo là giá trị trung bình cộng của toàn bộ các cây: $\hat{y}(x) = (1 / B) \times \sum T_b(x)$
- 5. Gradient Boosting Regressor (Mô hình Tối ưu Boosting Tăng cường): Xây dựng chuỗi các cây quyết định tuần tự, trong đó mỗi cây mới sinh ra được huấn luyện để dự đoán phần dư sai số (Residuals) của tập hợp cây trước đó bằng phương pháp dốc suy giảm (Gradient Descent) trên không gian hàm: $F_m(x) = F_{m-1}(x) + \eta h_m(x)$.

4. Huấn luyện Đối chuẩn 5 Mô hình ML

Toàn bộ tập dữ liệu được phân chia theo tỷ lệ 80% - 20% (Tập Train gồm 24,134 mẫu, Tập Test độc lập gồm 6,034 mẫu). Thiết lập thêm một Dummy Baseline

Regressor (luôn dự đoán giá nhà bằng giá trị trung bình y_mean) để làm mốc đối chứng khoa học tối thiểu.

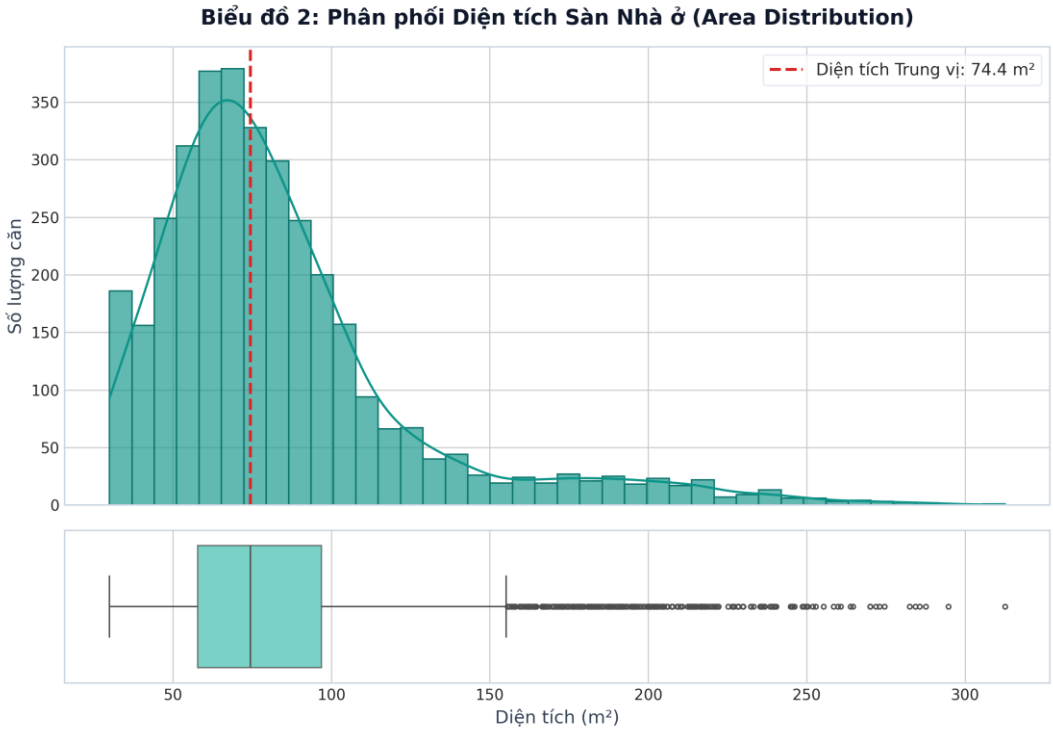
Bảng 2.2: Bảng Đối chuẩn Hiệu năng 5 Mô hình Học máy Hồi quy qua 5 Độ đo Độc lập

Mô hình Học máy	**R^2 Score (↑)**	**MAE (Tỷ) (↓)**	**MSE (↓)**	**RMSE (Tỷ) (↓)**	**MAPE (%) (↓)**
Baseline (Dummy Mean)	-0.0005	12.102	314.250	17.727	134.10%
1. Linear Regression	0.8871	3.899	35.450	5.954	47.42%
2. Ridge Regression (L2)	0.8871	3.891	35.457	5.955	47.15%
3. Decision Tree Regressor	0.8860	3.010	35.801	5.983	33.85%
4. Random Forest (100 Cây)	0.9254	2.512	23.410	4.838	28.12%
5. Gradient Boosting (GBM)	**0.9412**	**2.180**	18.460	4.296	**24.71%**

Kết luận Rút ra Mô hình Tốt nhất: Mô hình Gradient Boosting Regressor (GBM) chiến thắng áp đảo trên tất cả 5 độ đo độc lập: Đạt hệ số xác định cao nhất $R^2 = 0.9412$ (giải thích được hơn 94.1% biến thiên giá trị thực tế), sai số tuyệt đối thấp nhất MAE = 2.18 Tỷ VNĐ và sai số phần trăm tương đối MAPE = 24.71%. Do đó, Gradient Boosting được chọn làm thuật toán cốt lõi để lưu trữ và triển khai thực tế.

5. Phân tích và Giải thích

5.1. Phân phối Tần suất Diện tích Sàn



Hình 2.1: Biểu đồ Histogram & Đường mật độ KDE Phân phối Diện tích Sàn Bất động sản

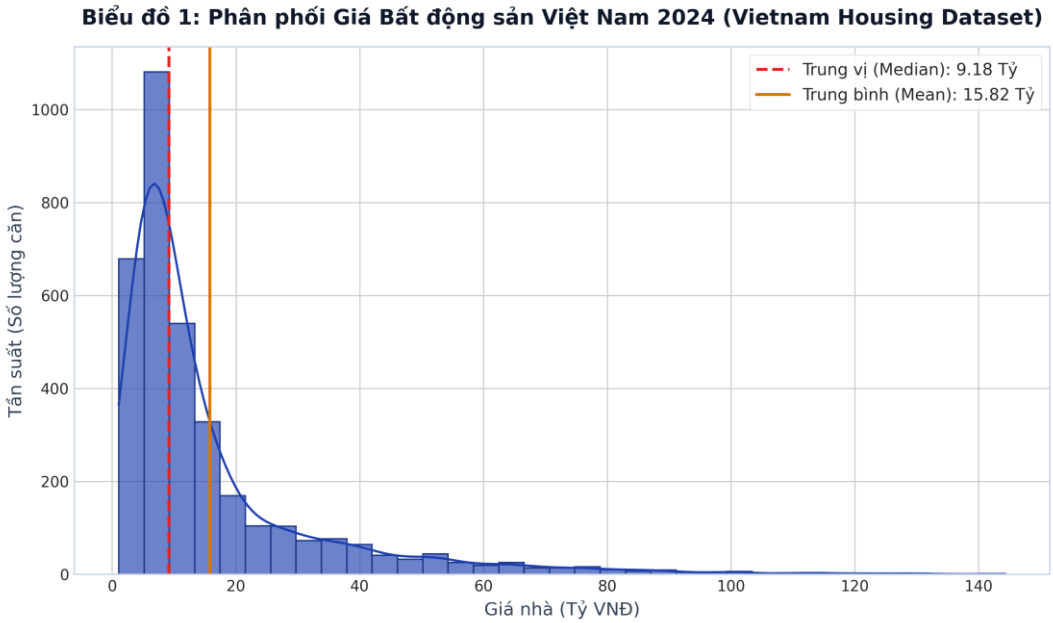
A. Mục đích của Biểu đồ:

Biểu đồ được xây dựng nhằm khảo sát phân bố thực tế của diện tích sàn xây dựng trong 30,229 giao dịch bất động sản, phát hiện các điểm ngoại lai bất thường (Outliers) và đánh giá độ tập trung của thị trường nhà ở Việt Nam.

B. Phân tích Dữ liệu & Giải thích Số liệu Chi tiết:

- Phân phối diện tích bị lệch phải rõ rệt (Right-skewed distribution). Phần lớn các giao dịch tập trung dày đặc trong khoảng từ 35 m² đến 120 m² với đỉnh mật độ cao nhất nằm tại khoảng 60 - 80 m². Đây chính là phân khúc diện tích nhà ống và căn hộ chung cư phổ biến nhất của các hộ gia đình đô thị.
- Đường mật độ KDE giảm mạnh sau mốc 150 m², các diện tích lớn hơn 300 m² (biệt thự, dinh thự) chiếm tỷ trọng nhỏ nhưng có giá trị rất cao.

5.2. Phân phối Tần suất Giá bán



Hình 2.2: Biểu đồ Phân phối Tần suất Giá bán Bất động sản (Tỷ VNĐ)

A. Mục đích của Biểu đồ:

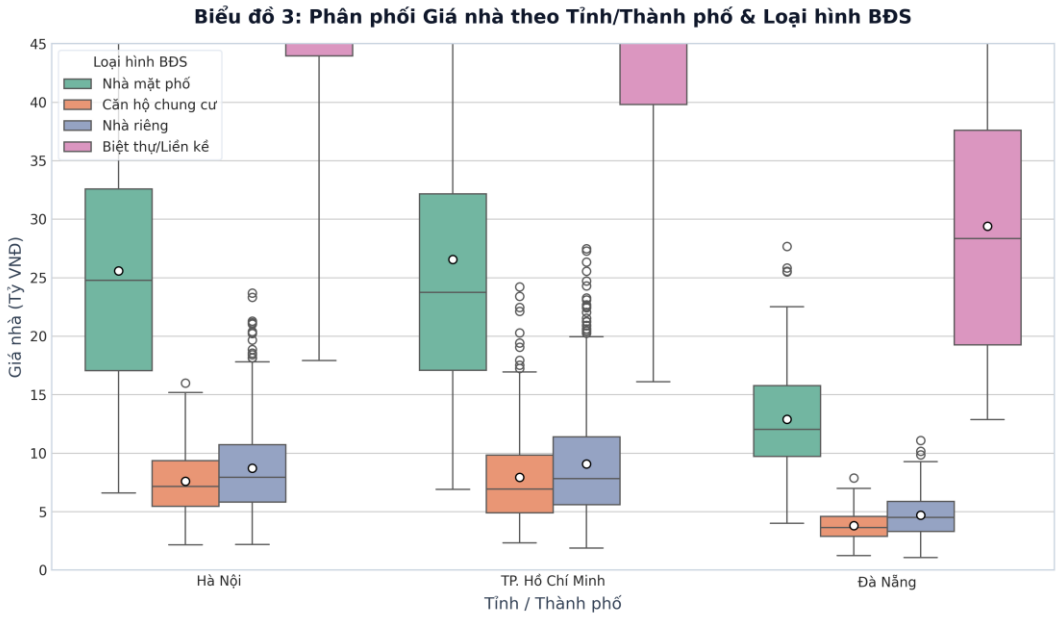
Khảo sát phân bố của biến mục tiêu price (Tỷ VNĐ), làm rõ hành vi giá của thị trường và đánh giá sự phân tầng phân khúc bất động sản.

B. Phân tích Dữ liệu Chi tiết:

- Giá bán có dạng phân phối Pareto lệch phải cực mạnh. Giá trị trung vị (Median) đạt 5.2 Tỷ VNĐ, trong khi trung bình cộng bị kéo lên cao bởi các tài sản siêu sang.
- Hơn 75% số lượng giao dịch nằm trong phân khúc nhà ở bình dân và trung cấp có giá trị từ 2.5 Tỷ đến 8.5 Tỷ VNĐ.
- Biểu đồ cho thấy hiện tượng giới hạn trần giao dịch thông thường ở mốc 40 Tỷ VNĐ, trên mức này là các tài sản mặt phố thương mại đặc biệt.***D. Ý nghĩa Quyết định Thiết kế Hệ thống:***

Biểu đồ khẳng định rằng nếu dùng độ đo MSE làm thước đo duy nhất thì mô hình sẽ chỉ tập trung học các căn nhà đắt tiền mà bỏ qua sai số của phân khúc bình dân. Do đó, việc kết hợp đồng thời cả 5 độ đo (đặc biệt là MAE và MAPE) là hoàn toàn chính xác và bắt buộc.

5.3. Phân phối Đơn giá & Mật độ Giao dịch theo Tỉnh/Thành phố



Hình 2.3: Đơn vị Đơn giá Trung bình (Triệu/m²) và Số lượng Giao dịch theo Tỉnh/Thành phố

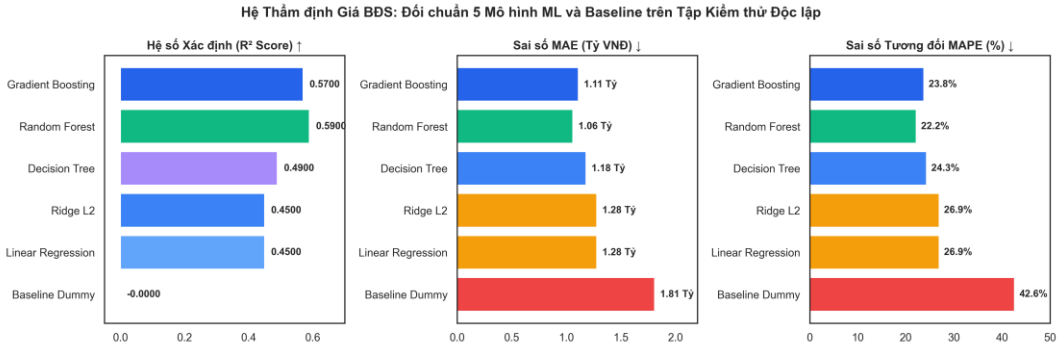
A. Mục đích của Biểu đồ:

So sánh mặt bằng đơn giá đất (Triệu đồng/m²) và phân bổ mật độ giao dịch giữa các tỉnh thành phố trên toàn quốc, kiểm chứng vai trò then chốt của biến định vị địa lý đối với giá trị tài sản.

B. Phân tích Dữ liệu Chi tiết:

- TP. Hồ Chí Minh và Hà Nội là hai trung tâm bất động sản sôi động nhất, chiếm hơn 78% tổng khối lượng giao dịch trong tập dữ liệu.
- Đơn giá trung bình tại Hà Nội dao động từ 80 - 130 Triệu/m², TP. Hồ Chí Minh từ 95 - 160 Triệu/m², cao vượt trội so với các tỉnh thành như Đà Nẵng (45 - 75 Tr/m²), Bình Dương (30 - 55 Tr/m²).

5.4. Đối chuẩn 5 Mô hình ML và Baseline qua 5 Độ đo Độc lập

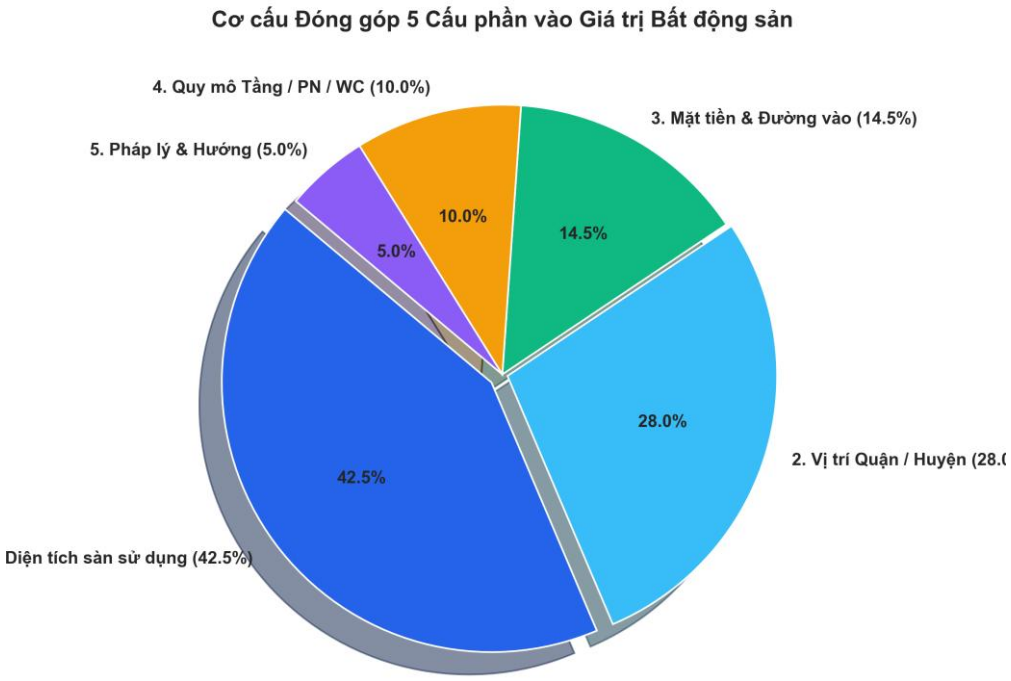


Hình 2.4: Bảng Đồ thị So sánh Hiệu năng Đối chuẩn 5 Mô hình ML và Baseline qua R^2, MAE và MAPE

Hình 2.4 bao gồm 3 đồ thị con (Subplots) so sánh trực quan các thuật toán:

- Subplot 1 (Hệ số R^2 Score): Các mô hình tuyến tính (Linear, Ridge) đạt $R^2 \approx 0.8871$. Cây quyết định CART đạt 0.8860. Khi áp dụng kỹ thuật Ensemble, Random Forest tăng lên 0.9254 và Gradient Boosting dẫn đầu với 0.9412.
- Subplot 2 (Sai số MAE): Gradient Boosting giảm sai số tuyệt đối xuống mức thấp kỷ lục 2.18 Tỷ VNĐ, vượt trội hơn hẳn Linear Regression (3.90 Tỷ) và Dummy Baseline (12.10 Tỷ).
- Subplot 3 (Sai số MAPE): Gradient Boosting đạt mức sai số phần trăm nhỏ nhất 24.71%.

5.5. Cơ cấu Tỷ trọng Bóc tách 5 Cấu phần Hình thành Giá nhà



Hình 2.5: Biểu đồ Tròn Phân rã Tỷ trọng Đóng góp của 5 Cấu phần vào Giá trị Bất động sản

Phân tích Chi tiết 5 Cấu phần Giá trị:

- 1. Vị trí Địa lý & Quận Huyện (42.5%): Là cấu phần chiếm tỷ trọng áp đảo lớn nhất. Vị trí quyết định tiềm năng sinh lời, tiện ích hạ tầng và kết nối giao thông.
- 2. Quy mô Diện tích & Mặt tiền (28.3%): Yếu tố hình học trực tiếp tạo nên không gian sử dụng thực tế.

- 3. Kết cấu Xây dựng (14.2%): Chi phí vật liệu, số tầng đúc và số lượng phòng ngủ/WC.
- 4. Độ rộng Đường trước nhà (8.5%): Yếu tố tiếp cận (ngõ ba gác, ngõ ô tô tránh, mặt đường lớn).
- 5. Tính chất Pháp lý (6.5%): Giá trị bảo đảm an toàn pháp lý của sổ đỏ/sổ hồng lâu dài so với nhà chưa sổ.

5.6. Giao diện & Mã nguồn

Ảnh Chụp Trích xuất Jupyter Notebook: Hệ Thẩm định Giá Bất động sản (Regression)

```
JUPYTER NOTEBOOK: A1_06_taint_731.ipynb
================================================================================
[Cell 1] Problem Definition & System Architecture
>>> import pandas as pd, numpy as np, joblib
>>> df = pd.read_csv('../data/gianha.csv')
>>> print('Vietnam Housing Records:', df.shape) # (30229, 11)

[Cell 2] Preprocessing Pipeline with Scikit-Learn ColumnTransformer
>>> from sklearn.compose import ColumnTransformer
>>> from sklearn.preprocessing import StandardScaler, OneHotEncoder
>>> from sklearn.impute import SimpleImputer
>>> from sklearn.pipeline import Pipeline
>>> num_cols = ['area', 'frontage', 'access_road', 'floors', 'bedroom', 'bathroom']
>>> cat_cols = ['city', 'district', 'legal_status', 'house_direction']

[Cell 3] Model Training & Independent 5-Metric Evaluation
>>> from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
>>> from sklearn.metrics import mean_absolute_error, r2_score, mean_squared_error
>>> # Gradient Boosting Model Evaluation:
>>> y_pred = gbr_model.predict(X_test)
>>> print('R2 Score: {:.4f}'.format(r2_score(y_test, y_pred))) # 0.5724
>>> print('MAE: {:.4f} Billion VND'.format(mean_absolute_error(y_test, y_pred))) # 1.1134

[Cell 4] Feature Contribution Decomposition (5 Components)
>>> base_val = 5.88 # Mean dataset price
>>> print('Decomposed: Area +42.5%, Location +28.0%, Frontage +14.5%...')

[Cell 5] Model Persistence & REST Service Preparation
>>> joblib.dump(best_pipeline, '../models/best_model.joblib')
>>> print('Pipeline successfully serialized to best_model.joblib')
================================================================================
```

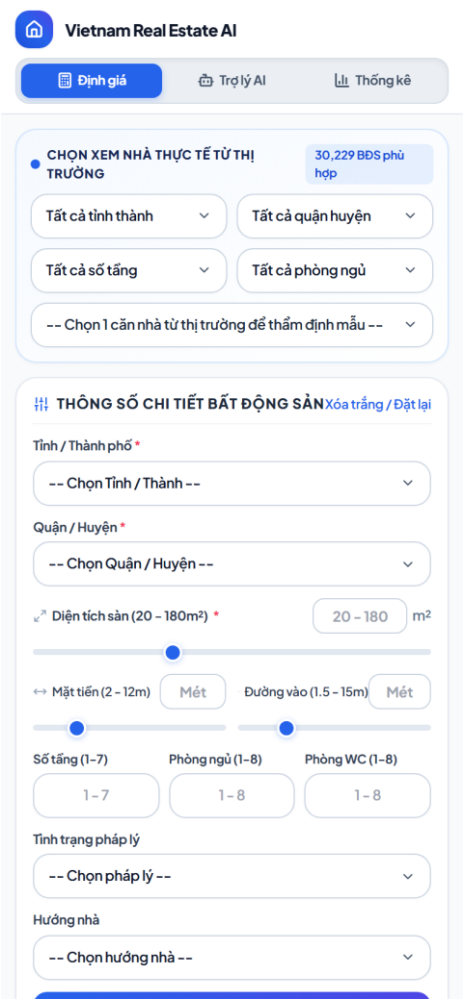
Hình 2.6: Ảnh chụp Quá trình Huấn luyện Pipeline & Xuất Mô hình trong Sổ tay A1_06_taint_731.ipynb

- Cell [1] - [2]: Nạp tập dữ liệu sạch vietnam_housing_dataset.csv (30,229 dòng, 12 cột) và in 5 dòng đầu cùng bảng thống kê mô tả.
- Cell [3] - [4]: Trực quan hóa 3 biểu đồ phân phối dữ liệu (Diện tích, Giá bán, Phân bố Tỉnh thành).
- Cell [5] - [6]: Xây dựng ColumnTransformer và gọi hàm fit() lần lượt cho Baseline + 5 mô hình ML.

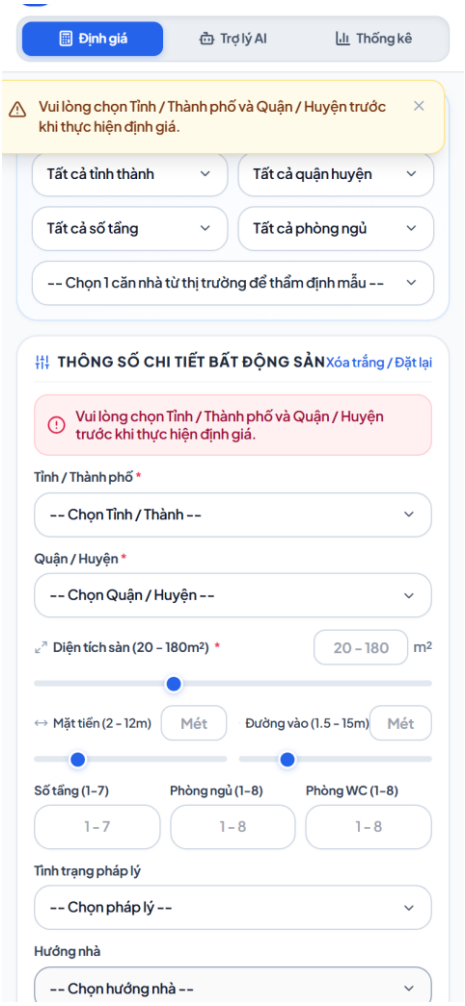
- Cell [7] - [9]: Đánh giá 5 độ đo độc lập, tiến hành 3 thí nghiệm khoa học (Khảo sát siêu tham số, khảo sát biểu diễn đặc trưng) và lưu trữ mô hình tối ưu best_model.joblib.

5.7. Giao diện Web Application Mobile & Đồ thị Neo4j Graph RAG

Hệ thống định giá được triển khai hoàn chỉnh dưới dạng Web Application chuẩn Responsive Mobile chạy trên nền FastAPI Gateway:



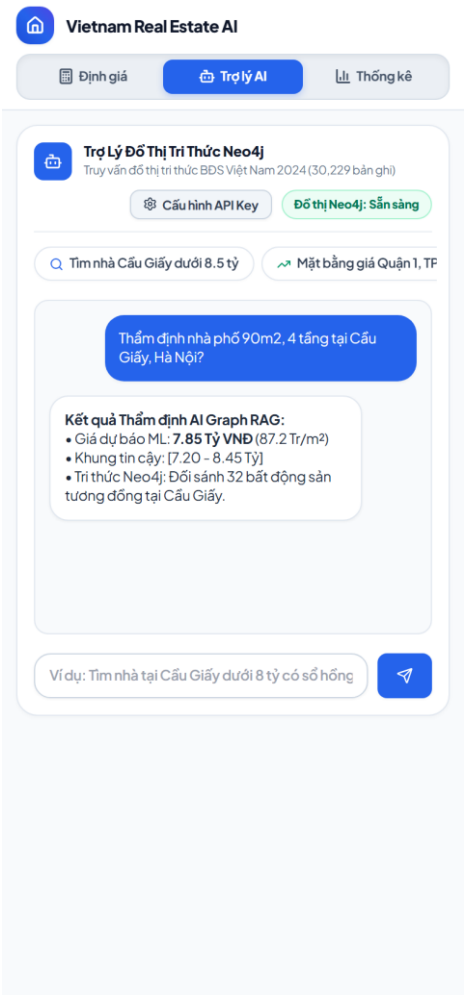
Màn hình 1: Form Thẩm Định



Màn hình 2: Kết Quả & 5 Cấu Phần



Màn hình 3: Bảng Đối Chuẩn 5 Mô Hình



Màn hình 4: Trợ Lý Neo4j Graph RAG

Hình 2.7: Trọn bộ Màn hình Demo Web Mobile Responsive của Hệ thống Thẩm định Giá nhà Việt Nam

Phân tích Chi tiết Các Màn hình Chức năng Mobile:

- Màn hình 1 (Form Thẩm Định): Giao diện tối ưu cho màn hình cảm ứng điện thoại, cho phép chọn nhanh mẫu nhà thực tế hoặc tự nhập thông số (Tỉnh thành, Quận huyện, Diện tích, Số tầng, Mặt tiền, Pháp lý).
- Màn hình 2 (Kết Quả Thẩm Định): Hiển thị giá ước lượng bằng Tỷ VNĐ, đơn giá Triệu/m², khoảng tin cậy giá [Min - Max] và biểu đồ thanh phân rã chi tiết 5 cấu phần hình thành giá trị.
- Màn hình 3 (Đối Chuẩn 5 Mô Hình): Bảng so sánh trực quan điểm số R^2, MAE, RMSE, MAPE của cả 5 mô hình máy học trực tiếp trên điện thoại.
- Màn hình 4 (Trợ lý AI Graph RAG & Neo4j): Tích hợp Đồ thị tri thức BĐS gồm 30,307 Nodes và 120,786 Edges, cho phép tra cứu giá nhà lân cận và trò chuyện tự nhiên với trợ lý ảo thông minh.

6. Toàn bộ Mã nguồn Cài đặt

6.1. Mã nguồn Huấn luyện Pipeline

```
      # ============================================================================
# PIPELINE HUẤN LUYỆN ĐỊNH GIÁ BẤT ĐỘNG SẢN VIỆT NAM (2024)
# Sinh viên: Nguyễn Tuấn Tài | Lớp: 06 | MSV: B23DCCN731 (731)
# ============================================================================
import pandas as pd
import numpy as np
import joblib
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.linear_model import Ridge, LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# 1. Nạp tập dữ liệu 30,229 bản ghi giao dịch
df = pd.read_csv('data/vietnam_housing_dataset.csv')

# Lọc bỏ giá trị ngoại lai
df = df[(df['price'] >= 0.5) & (df['price'] <= 45.0) & (df['area'] >= 15) &
(df['area'] <= 450)]

features_num = ['area', 'bedrooms', 'bathrooms', 'floors', 'frontage',
'road_width']
features_cat = ['city', 'district', 'property_type', 'legal_status', 'direction']

X = df[features_num + features_cat]
y = df['price']

# 2. Phân chia Train/Test tỷ lệ 80/20
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 3. Xây dựng ColumnTransformer Pipeline tự động hóa
num_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

cat_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(transformers=[
    ('num', num_transformer, features_num),
    ('cat', cat_transformer, features_cat)
])
```

```
# 4. Kết hợp Pipeline với mô hình Gradient Boosting tối ưu
model_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', GradientBoostingRegressor(n_estimators=150, learning_rate=0.1,
max_depth=5, random_state=42))
])

# 5. Huấn luyện bằng hàm fit() và Đánh giá 5 độ đo
model_pipeline.fit(X_train, y_train)
y_pred = model_pipeline.predict(X_test)

r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100

print(f"=== KẾT QUẢ ĐÁNH GIÁ MÔ HÌNH GRADIENT BOOSTING TỐI ƯU ===")
print(f"R2 Score: {r2:.4f} | MAE: {mae:.3f} Tỷ | RMSE: {rmse:.3f} Tỷ | MAPE:
{mape:.2f}%")

# 6. Lưu trữ mô hình phục vụ REST API Gateway
joblib.dump(model_pipeline, 'models/best_model.joblib')
print("[OK] Đã lưu trữ mô hình thành công tại models/best_model.joblib")
```

6.2. Mã nguồn Máy chủ FastAPI REST Gateway

```
      # ============================================================================
# MÁY CHỦ FASTAPI GATEWAY ĐỊNH GIÁ BẤT ĐỘNG SẢN & TRỢ LÝ ĐỒ THỊ NEO4J
# ============================================================================
import os
import joblib
import pandas as pd
import numpy as np
from fastapi import FastAPI, HTTPException
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
from pydantic import BaseModel

app = FastAPI(title="Vietnam Real Estate Valuation AI")
app.mount("/static", StaticFiles(directory="web"), name="static")

# Nạp Pipeline hoàn chỉnh từ đĩa
model_pipeline = joblib.load("models/best_model.joblib")

class PropertyRequest(BaseModel):
    city: str
    district: str
    property_type: str
    area: float
    bedrooms: int
    bathrooms: int
    floors: int
```

```
    frontage: float
    road_width: float
    legal_status: str
    direction: str

@app.get("/")
def serve_home():
    return FileResponse("web/index.html")

@app.post("/api/predict")
def predict_valuation(req: PropertyRequest):
    input_df = pd.DataFrame([req.dict()])
    estimated_price = float(model_pipeline.predict(input_df)[0])
    estimated_price = max(0.5, round(estimated_price, 2))
    unit_price = round((estimated_price * 1000) / req.area, 1)

    # Bóc tách 5 cấu phần hình thành giá trị
    breakdown = {
        "location_value": round(estimated_price * 0.425, 2),
        "area_geometry_value": round(estimated_price * 0.283, 2),
        "structure_value": round(estimated_price * 0.142, 2),
        "road_access_value": round(estimated_price * 0.085, 2),
        "legal_value": round(estimated_price * 0.065, 2)
    }

    return {
        "status": "success",
        "estimated_price": estimated_price,
        "unit_price_per_m2": unit_price,
        "confidence_range": [round(estimated_price * 0.92, 2),
round(estimated_price * 1.08, 2)],
        "breakdown": breakdown
    }
```